Конкурентность, парадигмы и управление памятью

ZIG

Новые языки программирования

Команда 16
Гуцалов Егор · Крутянский Роман · Медведьков Роман
Группа 5030102/30201

МАРШРУТ

Пять тем третьей части

Конкурентность и параллелизм

Объекты, функции и интерфейсы

Специфические конструкции Zig

Нативное выполнение без обязательной VM

Память, аллокаторы и время жизни

Примеры для Zig 0.17

Zig 0.17

Все примеры используют стандартную библиотеку

Полные программы находятся в папке src

Конкурентность и параллелизм

Конкурентность: задачи могут продвигаться независимо

Параллелизм: вычисления идут одновременно

Поток ОС может выполняться на отдельном ядре

Несколько задач могут делить один поток

Поток ОС: spawn и join

Делим данные на две части

Рабочий поток пишет только в left

Основной поток вычисляет right

Читаем общий результат после join

01_threads.zig

```
const data = [_]u32{ 1, 2, 3, 4, 5, 6 };
var left: u32 = 0;
var right: u32 = 0;
{
    const t = try std.Thread.spawn(
        .{}, sum, .{ data[0..3], &left },
    );
    defer t.join();
    sum(data[3..], &right);
}
std.debug.assert(left + right == 21);
```

```
Результат: sum=21
```

Общее состояние и синхронизация

value += 1 состоит из нескольких действий

Mutex защищает всю критическую секцию

Оба потока используют один mutex

После join счетчик равен 20 000

02_sync.zig

```
const Counter = struct {
    mutex: std.Io.Mutex = .init,
    value: u32 = 0,

    fn increment(self: *Counter, io: std.Io) void {
        self.mutex.lockUncancelable(io);
        defer self.mutex.unlock(io);
        self.value += 1;
    }
};
```

```
Результат: counter=20000
```

Атомики и порядок памяти

fetchAdd выполняет неделимое увеличение

monotonic подходит для независимого счетчика

Публикация других данных требует отдельного протокола

Атомики не заменяют mutex для сложных инвариантов

02_sync.zig

```
    var atomic = std.atomic.Value(u32).init(0);
    _ = atomic.fetchAdd(1, .monotonic);
    std.debug.assert(atomic.load(.monotonic) == 1);
```

```
Результат: atomic=1
```

Задачи через std.Io

async допускает выполнение прямо при вызове

concurrent требует доступной конкурентности

await получает результат задачи

Реализация Io определяет стратегию выполнения

03_tasks.zig

```
    const io = init.io;
    var first = io.async(square, .{12});
    // Also await first if concurrent fails.
    defer _ = first.await(io);
    var second = try io.concurrent(square, .{5});
    const b = second.await(io);
    const a = first.await(io);
    std.debug.assert(a + b == 169);
```

```
Результат: tasks=169
```

Какие стили поддерживает Zig

Процедурный: функции и явный поток управления

Объектный: структуры, методы, композиция

Функциональные приемы: функции как параметры

Полиморфизм: специализация или явная диспетчеризация

Структуры, методы и композиция

Player содержит Position как поле

Метод получает явный self: *Player

Вызов player.move(3) изменяет объект

Наследования классов и скрытой vtable нет

04_objects.zig

```
const Position = struct { x: i32 = 0, y: i32 = 0 };
const Player = struct {
    position: Position = .{},
    score: u32 = 0,

    fn move(self: *Player, dx: i32) void {
        self.position.x += dx;
    }
};
```

```
Результат: x=3
```

Статический интерфейс через anytype

Тип аргумента определяется при компиляции

Операция area() задает требование к типу

Square и Rectangle реализуют одно соглашение

Неподходящий тип вызовет ошибку компиляции

05_protocols.zig

```
const Square = struct {
    side: u32,
    fn area(self: Square) u32 {
        return self.side * self.side;
    }
};
fn areaOf(shape: anytype) u32 {
    return shape.area();
}
```

Результат: areaOf(Square{ .side = 3 }) = 9

Выбор реализации во время выполнения

union(enum) хранит один из вариантов и тег

switch выбирает реализацию по тегу

Компилятор проверяет полноту switch

Набор вариантов известен заранее

05_protocols.zig

```
const Shape = union(enum) {
    square: Square,
    rectangle: Rectangle,

    fn area(self: Shape) u32 {
        return switch (self) {
            .square => |s| s.area(),
            .rectangle => |r| r.area(),
        };
    }
};
```

```
Результат: static=9, dynamic=19
```

Функции высшего порядка

map получает функцию преобразования

comptime фиксирует операцию при компиляции

Входной срез доступен только для чтения

Выходной буфер предоставляет вызывающий код

06_functions.zig

```
fn square(x: i32) i32 { return x * x; }

fn map(input: []const i32,
    output: []i32,
    comptime f: fn (i32) i32) void
{
    std.debug.assert(input.len == output.len);
    for (input, output) |x, *y| y.* = f(x);
}
```

```
Результат: map={ 1, 4, 9 }
```

comptime: вычисления при компиляции

Обычная функция строит таблицу квадратов

comptime требует выполнить вызов заранее

В программе остается готовый результат

Память компилятора нельзя передать в runtime

07_comptime.zig

```
fn makeSquares() [4]u32 {
    var table: [4]u32 = undefined;
    for (&table, 0..) |*v, i| v.* = @intCast(i * i);
    return table;
}

const table = comptime makeSquares();
```

```
Результат: table={ 0, 1, 4, 9 }
```

Рефлексия и inline for

std.meta.fieldNames получает имена полей типа

inline for разворачивает цикл при компиляции

Тело специализируется для каждого имени

Печать остается операцией времени выполнения

07_comptime.zig

```
const Config = struct { port: u16, debug: bool };

    const names = comptime std.meta.fieldNames(Config);
    inline for (names) |name| {
        std.debug.print("field={s}\n", .{name});
    }
```

```
Результат: field=port
field=debug
```

Управление представлением данных

packed struct(u8) задает битовое представление

Поля занимают 1 + 3 + 4 бита

@bitCast сохраняет биты при смене типа

extern struct применяет правила внешнего ABI

08_layout_simd.zig

```
const Flags = packed struct(u8) {
    ready: bool,
    mode: u3,
    reserved: u4 = 0,
};

    const flags = Flags{ .ready = true, .mode = 3 };
    const raw: u8 = @bitCast(flags);
    std.debug.assert(raw == 7);
```

```
Результат: flags=7
```

Векторные операции

@Vector описывает несколько однотипных элементов

Оператор + действует поэлементно

@reduce сворачивает вектор в одно значение

Машинные инструкции зависят от цели и оптимизации

08_layout_simd.zig

```
const a: @Vector(4, i32) = .{ 1, 2, 3, 4 };
const b: @Vector(4, i32) = @splat(10);
const result = a + b;
std.debug.assert(@reduce(.Add, result) == 50);
```

```
Результат: vector={ 11, 12, 13, 14 }
```

Нативный код без обязательной VM

Компилятор создает код для выбранной платформы

Запуск нативного файла не требует JVM или CLR

comptime выполняет компилятор до запуска

WebAssembly требует подходящей среды выполнения

09_native.zig

```
const builtin = @import("builtin");

pub fn main() void {
    std.debug.print(
        "target={s}-{s}, mode={s}\n", .{
        @tagName(builtin.cpu.arch),
        @tagName(builtin.os.tag),
        @tagName(builtin.mode),
    });
}
```

```
Результат: target=x86_64-windows, mode=debug
```

Режимы сборки и проверки

Debug: проверки безопасности и быстрая отладка

ReleaseSafe: оптимизация с проверками безопасности

ReleaseFast: оптимизация, проверки обычно отключены

ReleaseSmall: приоритет размеру, проверки обычно отключены

Значения, указатели и время жизни

Нет автоматического сборщика мусора

Срез хранит указатель и длину, без владения

const запрещает запись через данное представление

Время жизни и освобождение контролирует программист

Allocator как параметр функции

makeData получает стратегию выделения памяти

Результат передает владение вызывающему коду

free использует тот же allocator

defer связывает освобождение с областью видимости

10_memory.zig

```
    var debug_allocator:
        std.heap.DebugAllocator(.{}) = .init;
    defer std.debug.assert(
        debug_allocator.deinit() == .ok,
    );
    const a = debug_allocator.allocator();
    const data = try makeData(a);
    defer a.free(data);
```

```
Результат: data={ 42, 42, 42, 42 }
```

Арена и фиксированный буфер

ArenaAllocator освобождает группу объектов целиком

Арена подходит для одного запроса или этапа

FixedBufferAllocator берет память из готового буфера

После deinit арены все ее данные недействительны

11_allocators.zig

```
    var arena = std.heap.ArenaAllocator.init(
        std.heap.page_allocator,
    );
    defer arena.deinit();
    const a = arena.allocator();
    const name = try a.dupe(u8, "Zig");
    const values = try a.alloc(u32, 3);
    @memset(values, 7);
    std.debug.assert(values[2] == 7);
```

```
Результат: arena=Zig, values={ 7, 7, 7 }
```

Проверка утечек и отказа выделения

std.testing.allocator отслеживает утечки в тесте

failing_allocator моделирует OutOfMemory

Проверяем результат и освобождение ресурса

Тесты не заменяют анализ времени жизни

10_memory.zig

```
test "allocation failure propagates" {
    try std.testing.expectError(error.OutOfMemory,
        makeData(std.testing.failing_allocator));
}
```

```
Результат: All 2 tests passed.
```

ИТОГИ

Что определяет стиль Zig

Конкурентность требует выбора механизма и синхронизации

Поведение описывают функциями, структурами и явными интерфейсами

comptime переносит часть работы в компилятор

Нативное выполнение сочетается с явным управлением памятью

Документация и исходники

Zig 0.17 Language Reference
ziglang.org/documentation/0.17.0/

Стандартная библиотека Zig
github.com/ziglang/zig/tree/0.17.0/lib/std

https://ziglang.org/overview/